



**PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS**

React Props y estilos CSS

Presentado a: Instructora Alexandra Guevara
Por Aprendiziz: Cristian David Yalanda
Ficha: 3312932
Grupo: ADSO

Tecnólogo en Análisis y Desarrollo de Software
Servicio Nacional de Aprendizaje SENA
Centro de Teleinformática y Producción Industrial
Regional Cauca

Popayán, día **24** de **09** del año **2026**



PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

Tabla de Contenido

Contenido

Actividad o APLICACIÓN DE CONOCIMIENTOS.....	3
1. Enunciado.....	3
2. Situación problémica.....	3
3. Objetivo	3
4. Requerimientos del desarrollo	3
5. Estructura sugerida de componentes.....	3
6. Uso obligatorio de Props.....	4
7. Maquetado que debe ser recreado	4
8. Aplicación de CSS	4
9. Condiciones de entrega.....	5
10. Reto de aplicación	5
11. Preguntas de aplicación.....	5
1. Introducción y desarrollo paso a paso.....	6
Paso 1: Creación del proyecto	6
Paso 2: División de la interfaz en componentes	7
Paso 3: Comunicación mediante props	8
Paso 4: Estilos con CSS.....	8
Paso 5: Reto de aplicación (prop destacado)	8
Paso 6: Ajuste fino comparando con la imagen de referencia	9
2. Resultado final de la interfaz	9
3. Estructura de componentes propuesta y justificación.....	10
4. Respuestas a las preguntas de aplicación	10



PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL FORMATO ENTREGA DE EVIDENCIAS

Actividad o APLICACIÓN DE CONOCIMIENTOS

1. Enunciado

Una empresa requiere desarrollar la interfaz inicial de un sistema web administrativo. El equipo de desarrollo ha recibido como referencia visual el maquetado presentado en la imagen. El aprendiz deberá construir esta interfaz utilizando ReactJS, aplicando componentes reutilizables, comunicación entre componentes mediante props y estilos CSS.

El propósito del ejercicio no es copiar una imagen mediante una sola estructura HTML, sino analizar la interfaz, identificar sus diferentes partes y convertirlas en componentes de React. Los datos que cambian entre las tarjetas deberán ser enviados mediante props desde el componente padre hacia los componentes hijos.

2. Situación problemática

La empresa necesita una pantalla de inicio para mostrar información de diferentes servicios o productos. La interfaz debe permitir reutilizar las mismas tarjetas para presentar información diferente sin crear un componente independiente para cada tarjeta.

A partir de la imagen de referencia, el aprendiz deberá identificar la estructura visual y desarrollar una versión funcional y organizada en ReactJS.

3. Objetivo

Aplicar los conocimientos sobre componentes padre e hijo, props y CSS mediante la construcción de una interfaz tipo Dashboard en ReactJS, utilizando un diseño basado en la imagen de referencia.

4. Requerimientos del desarrollo

- Crear el proyecto utilizando ReactJS.
- Construir un componente principal App que organice la interfaz.
- Crear un componente Sidebar para representar la barra lateral izquierda.
- Crear un componente Dashboard para representar el área principal.
- Crear un componente Card o Tarjeta reutilizable para representar las tarjetas de información.
- Utilizar props para enviar información desde el componente padre hacia los componentes hijos.
- Utilizar múltiples props en las tarjetas, no solamente una.
- Crear mínimo seis tarjetas utilizando el mismo componente reutilizable.
- Aplicar estilos mediante archivos CSS externos.
- Organizar la interfaz utilizando Flexbox y/o CSS Grid.
- Implementar colores, bordes, espaciados, tamaños y distribución similares a la imagen de referencia.
- Agregar efectos hover en elementos interactivos.
- Mantener una estructura de carpetas organizada.

5. Estructura sugerida de componentes

El aprendiz deberá proponer la estructura de componentes utilizada y en un documento indicar el porque.



PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL FORMATO ENTREGA DE EVIDENCIAS

6. Uso obligatorio de Props

El componente Card deberá recibir información mediante varias props. Como mínimo, cada tarjeta deberá recibir los siguientes datos:

- titulo
- descripcion
- categoria
- imagen
- precio o valor
- estado

Ejemplo de uso conceptual:

```
<Card
  titulo="Diseño Web"
  descripcion="Diseño de interfaces modernas"
  categoria="Diseño"
  imagen="diseno.jpg"
  precio={250000}
  estado="Disponibile"
/>
```

El ejemplo anterior solamente representa la forma esperada de enviar información. El aprendiz deberá definir la implementación completa del componente.

7. Maquetado que debe ser recreado

La interfaz deberá tomar como referencia directa la imagen suministrada. Se deben identificar y recrear, como mínimo, los siguientes elementos:

- Barra lateral vertical ubicada al lado izquierdo.
- Botón o elemento superior del menú.
- Elementos de navegación dentro de la barra lateral.
- Área principal de contenido.
- Encabezado del Dashboard.
- Barra o elementos de navegación/filtros superiores.
- Contenedor principal de tarjetas.
- Seis tarjetas distribuidas en una cuadrícula de dos o tres columnas según el tamaño disponible.
- Panel lateral derecho con información adicional.
- Botones o elementos de acción.
- Espaciado y distribución visual semejantes a la referencia.

8. Aplicación de CSS

La apariencia de la aplicación deberá construirse utilizando CSS. El aprendiz deberá aplicar selectores y propiedades apropiadas para obtener una interfaz visualmente similar a la referencia.

- display: flex y/o display: grid



PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL FORMATO ENTREGA DE EVIDENCIAS

- width y height
- margin y padding
- border-radius
- border

9. Condiciones de entrega

- La interfaz debe ser funcional y ejecutarse correctamente.
- No se permite crear un componente diferente para cada tarjeta.
- Las tarjetas deben reutilizar el mismo componente mediante diferentes props.
- Cada tarjeta debe recibir mínimo seis props.
- El contenido de las tarjetas debe ser diferente.
- Los estilos deben estar separados del JSX mediante archivos CSS.
- La estructura debe ser clara y organizada.
- La interfaz debe guardar una relación visual evidente con la imagen de referencia.
- El código debe presentar nombres de componentes y variables claros.
- Documento Word indicando el paso a paso (utilice capturas de pantalla) con sus propias palabras y la resolución del ejercicio.
- Entregue enlace de repositorio GITHUB o GITLAB donde se visualice el ejercicio y el documento.

10. Reto de aplicación

Agregar una prop denominada destacado. Cuando su valor sea true, la tarjeta deberá presentar una característica visual adicional, por ejemplo un borde, una etiqueta o un indicador que permita identificarla como destacada.

```
<Card  
  titulo="Servicio Premium"  
  descripcion="Servicio especializado"  
  categoria="Premium"  
  imagen="premium.jpg"  
  precio={450000}  
  estado="Disponible"  
  destacado={true}  
>
```

11. Preguntas de aplicación

1. ¿Cuál es el componente padre principal de la aplicación?
2. ¿Qué componentes pueden considerarse hijos de App?
3. ¿Cuál es la función del componente Card?
4. ¿Por qué es conveniente reutilizar un solo componente Card?
5. ¿Qué información se está enviando mediante props?
6. ¿Qué diferencia existe entre modificar el contenido mediante props y modificar su apariencia mediante CSS?



PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL FORMATO ENTREGA DE EVIDENCIAS

7. ¿Cómo se podría cambiar el diseño de todas las tarjetas modificando solamente el archivo CSS?
8. ¿Cómo se podría hacer que una tarjeta tenga una apariencia diferente utilizando una prop?
9. ¿Qué ventajas tiene dividir la interfaz en componentes?
10. ¿Qué mejoras realizaría para convertir este maquetado en una interfaz responsive?

1. Introducción y desarrollo paso a paso

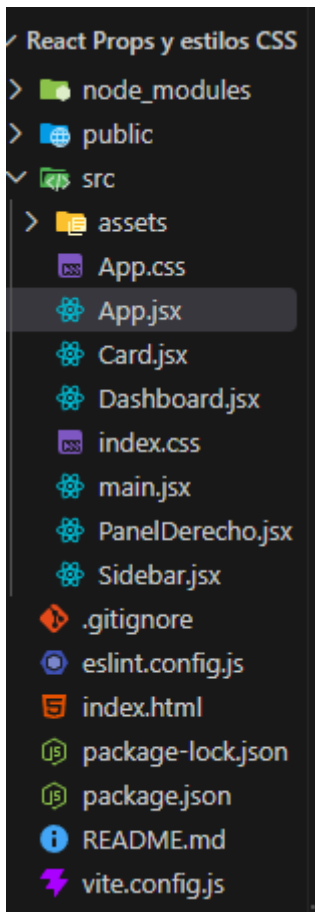
A continuación, explico, con mis propias palabras, cómo desarrollé la interfaz tipo Dashboard solicitada, aplicando creación de componentes, props y CSS en ReactJS

Paso 1: Creación del proyecto

Creé el proyecto de React utilizando Vite, lo que generó automáticamente la estructura básica con la carpeta src, el archivo main.jsx (que monta la aplicación en el DOM), el archivo App.jsx (componente principal) y los archivos de estilos. Dentro de src ubiqué cada componente (Sidebar.jsx, Dashboard.jsx, Card.jsx, PanelDerecho.jsx) directamente en la misma carpeta, sin subcarpetas adicionales. Todo el CSS del proyecto lo concentré en solo dos archivos: index.css, con las variables globales de color y el reset general, y App.css, con los estilos específicos de cada componente organizados por secciones



PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL FORMATO ENTREGA DE EVIDENCIAS



Paso 2: División de la interfaz en componentes

Analicé la imagen de referencia y en lugar de construir todo en un solo archivo, identifiqué cuatro partes claramente diferenciadas de la interfaz y creé un componente para cada una:

- Sidebar: la barra lateral izquierda de color amarillo con los íconos de navegación
- Dashboard: el área principal, que contiene el encabezado, los filtros y la cuadrícula de tarjetas
- Card: la tarjeta reutilizable de servicio, usada 6 veces con información diferente
- PanelDerecho: el panel lateral derecho con el detalle del servicio seleccionado

Estos componentes se importan y se ensamblan dentro de App.jsx, que es el componente padre principal de toda la aplicación



PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL FORMATO ENTREGA DE EVIDENCIAS

Paso 3: Comunicación mediante props

En App.jsx definí un arreglo llamado servicios con la información de los 6 servicios (título, descripción, categoría, imagen, precio, estado y destacado). Ese arreglo se envía como prop al componente Dashboard, y Dashboard lo recorre con map() para renderizar el componente Card seis veces, enviando cada vez las props individuales a cada tarjeta. Así, un mismo componente Card se reutiliza con datos distintos en cada llamado

```
<Card  
  titulo={servicio.titulo}  
  descripcion={servicio.descripcion}  
  categoria={servicio.categoria}  
  imagen={servicio.imagen}  
  precio={servicio.precio}  
  estado={servicio.estado}  
  destacado={servicio.destacado}  
>
```

Dentro de Card.jsx, esas props se reciben por desestructuración en el parámetro de la función y se muestran directamente en el JSX (título, descripción, categoría, precio formateado y estado con color según su valor)

Paso 4: Estilos con CSS

El JSX de cada componente está separado por completo de los estilos: ningún componente importa un CSS propio, sino que todos comparten dos archivos comunes. En index.css definí las variables globales (colores, radios de borde, sombra) y el reset general del proyecto; en App.css escribí las reglas de cada componente, agrupadas por secciones comentadas (Sidebar, Dashboard, Card, Panel derecho) para mantener el archivo ordenado. Usé className en vez de class, Flexbox para organizar la barra lateral y el layout general, y CSS Grid para distribuir las seis tarjetas en una cuadrícula de 3 columnas. También apliqué border-radius, padding, margin, colores y una transición para el efecto hover en las tarjetas y en los botones

Paso 5: Reto de aplicación (prop destacado)

Agregué la prop booleana destacado al componente Card. Cada tarjeta muestra en su imagen un pequeño indicador cuadrado (badge) en la esquina superior derecha, igual que en la imagen de referencia. Cuando destacado es true, ese badge cambia de color gris a amarillo mediante una clase CSS condicional (card-badge-activo), permitiendo distinguir visualmente las tarjetas destacadas sin alterar el resto del diseño

```
function Card({ titulo, descripcion, categoria, imagen,  
               precio, estado, destacado }) {  
  return (
```



PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL FORMATO ENTREGA DE EVIDENCIAS

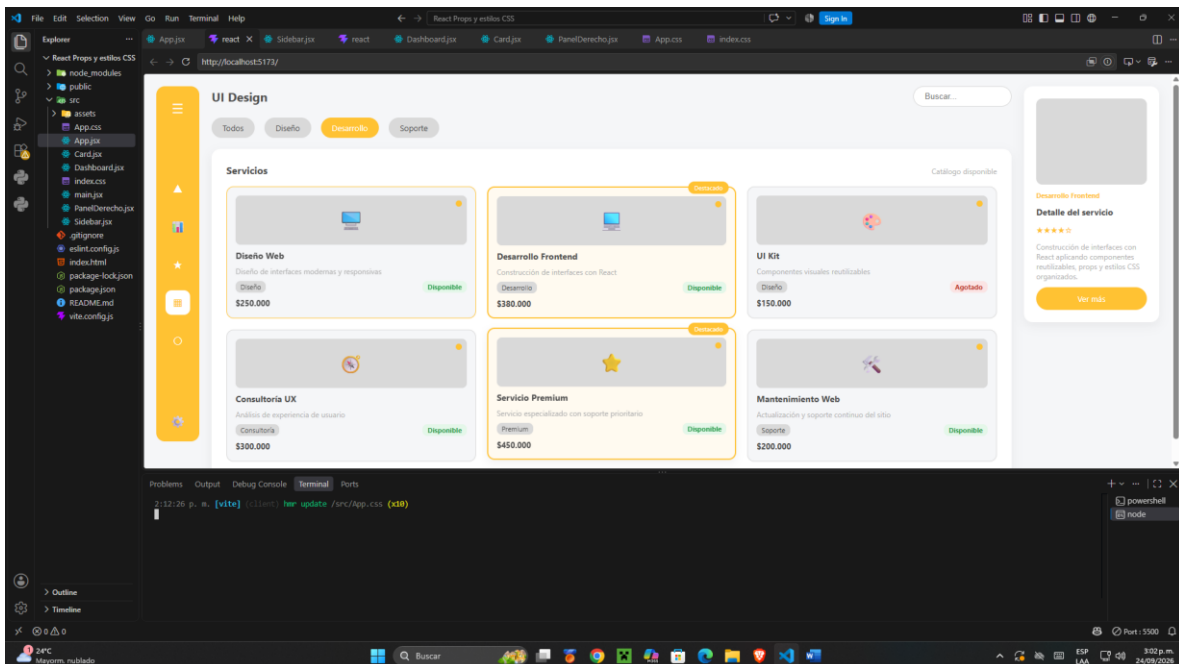
```
<div className="card">
  <div className="card-imagen">
    <span
      className={`card-badge ${destacado ? "card-badge-activo" : ""}`}
    />
  </div>
</div>
```

Paso 6: Ajuste fino comparando con la imagen de referencia

Después de una primera versión, comparé el resultado directamente contra la imagen del maquetado y corregí varios detalles para que la relación visual fuera más evidente: el indicador de las tarjetas pasó de un punto a un cuadrado (badge), el ícono inferior del Sidebar se cambió por un círculo, se agregó una segunda fila de sub-filtros (Recientes, Populares, Precio, Estado) dentro del panel blanco, y se añadió una fila adicional de tarjetas parcialmente visible en la parte inferior, simulando contenido cortado por scroll tal como aparece en la referencia

2. Resultado final de la interfaz

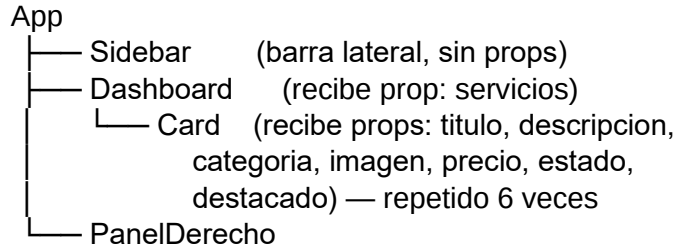
Captura de la aplicación en funcionamiento, ejecutada con npm run dev, mostrando la barra lateral, el encabezado con buscador y filtros, las seis tarjetas generadas a partir del componente Card reutilizable (dos de ellas marcadas como destacadas) y el panel lateral derecho





PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL FORMATO ENTREGA DE EVIDENCIAS

3. Estructura de componentes propuesta y justificación



Elegí esta estructura porque separa claramente responsabilidades: App concentra los datos y organiza el layout general; Sidebar y PanelDerecho son piezas visuales fijas que no necesitan recibir información variable; Dashboard actúa como intermediario que distribuye los datos del arreglo de servicios; y Card es el único componente que se repite, evitando así crear seis componentes distintos para seis tarjetas. Esta división facilita el mantenimiento: si se necesita otra tarjeta, solo se agrega un objeto más al arreglo, sin tocar el JSX

4. Respuestas a las preguntas de aplicación

1. ¿Cuál es el componente padre principal de la aplicación?

App.jsx, ya que es el que organiza y renderiza a los demás componentes (Sidebar, Dashboard y PanelDerecho)

2. ¿Qué componentes pueden considerarse hijos de App?

Sidebar, Dashboard y PanelDerecho son hijos directos de App. Card es hijo de Dashboard (nieto de App)

3. ¿Cuál es la función del componente Card?

Representar visualmente la información de un servicio (título, descripción, categoría, imagen, precio y estado) de forma reutilizable, mostrando datos distintos según las props que recibe

4. ¿Por qué es conveniente reutilizar un solo componente Card?

Porque evita duplicar código: en lugar de crear seis componentes casi idénticos, se usa uno solo con datos diferentes mediante props. Esto hace el proyecto más corto, más fácil de mantener y de escalar (agregar una tarjeta nueva no requiere código adicional)

5. ¿Qué información se está enviando mediante props?

El título, la descripción, la categoría, la imagen, el precio, el estado y el valor booleano destacado de cada servicio



PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL FORMATO ENTREGA DE EVIDENCIAS

6. ¿Qué diferencia existe entre modificar el contenido mediante props y modificar su apariencia mediante CSS?

Las props cambian el contenido o comportamiento del componente (qué datos muestra o cómo se comporta), mientras que el CSS cambia únicamente su apariencia visual (colores, tamaños, bordes, espaciados) sin alterar la información que se muestra

7. ¿Cómo se podría cambiar el diseño de todas las tarjetas modificando solamente el archivo CSS?

Como todas las tarjetas comparten las mismas clases (card, card-titulo, card-precio, etc.), basta con modificar esas reglas una sola vez dentro de la sección "Card" de App.css para que el cambio se refleje automáticamente en las seis tarjetas

8. ¿Cómo se podría hacer que una tarjeta tenga una apariencia diferente utilizando una prop?

Enviando una prop (por ejemplo destacado) y usando su valor dentro del componente para agregar condicionalmente una clase CSS distinta, tal como se hizo en el reto de aplicación

9. ¿Qué ventajas tiene dividir la interfaz en componentes?

Facilita la lectura y el mantenimiento del código, permite reutilizar piezas de interfaz, hace más sencillo detectar y corregir errores, y permite que varias personas trabajen en distintos componentes al mismo tiempo

10. ¿Qué mejoras realizaría para convertir este maquetado en una interfaz responsive?

Usaría media queries para reducir la cantidad de columnas del grid en pantallas pequeñas, ocultaría o convertiría el Sidebar en un menú colapsable, apilaría el panel derecho debajo del contenido principal en móviles y usaría unidades relativas (% , rem , vw) en lugar de tamaños fijos